

AENGM0074 — Individual Reflective Report D7 GOLDMINE (Unlimited)

Dmytro Chuprynyuk (Apollo)

University of Bristol
MSc Robotics

April 2026

Note: This is the comprehensive, unlimited-page goldmine document.
The final 5-page submission will be compressed from this source.

Word target: 6000–8000 words across 4 sections.
All 10 sub-questions answered with full depth.

My role

1a. Roles and responsibilities — explicit, implicit, and how they shifted

After the first team meeting we looked at every task the project needed and split them up explicitly. Harish took hardware setup and communications between components. Zian took path planning. Robin took the flight state machine. Edward and I shared computer vision — I delegated the CV model training to Edward while I focused on the detection-to-GPS pipeline and integration with the rest of the system. These allocations were discussed and agreed in that first meeting; nobody was named as a single technical lead.

That was the formal picture. What actually happened was broader.

My explicit responsibility was computer vision. In practice I ended up building the full CV pipeline — detection to GPS coordinate conversion using calibration and altitude compensation, with pitch and roll correction because the drone's attitude affects coordinate accuracy — plus a complete simulation framework, a progressive test ladder for building confidence before each flight step, a safety-critical layer with geofence enforcement and zone repulsive fields, a web-based ground station that ran headless on the Pi, camera calibration in the lab (field of view, focal length measurements), post-processing and video analysis tools for the one real flight video we had, multiple dashboards and monitoring tools for live comparison of different vision models, and a formal complaint letter to the course organiser about the hardware situation. None of that was in my original brief.

The implicit roles accumulated one task at a time, whenever I saw a gap between where we were and a working demo. I didn't sit down and decide to take everything on; it just happened. When hardware wasn't working, I'd think "I'll build this simulation so we can keep moving." When we needed to calibrate the camera, I'd think "I'll write a calibration script." When Robin needed GPS coordinates for his state machine, I'd think "I'll build the interface." Each task made sense individually. Collectively, they meant I was doing far more than my share, and I didn't flag that to the team early enough.

Looking back through Tuckman's (1965) developmental sequence, the role drift maps onto the Forming-to-Storming transition: individual differences in working style start showing, but the team hasn't yet confronted them openly. Robin and I were the proactive members — when something was blocked, we'd find another angle and keep going. Edward and Zian were less proactive; they'd do what they were assigned, and do it conscientiously, but they didn't drive things forward on their own initiative. That's fair enough as a working style, but the mismatch meant the workload distribution was uneven from early on, and nobody called a meeting to address it. I certainly didn't. I just kept building.

The roles shifted in two visible ways. The first shift was slow and silent: as the two practice flight days came and went with faulty hardware (provided to us, not hardware we'd built), the project's centre of gravity migrated from the test field to my laptop. The laptop was the only place where forward progress was possible — I could run SITL simulation while the actual drone sat in a box. Nobody declared this shift; it just accumulated. During the failed flight days, the whole team felt demotivated — me included. I'd had loads of things prepared: an in-depth simulation I wanted to test, a bunch of test scripts I wanted to run. But instead of just being frustrated, I decided to be proactive: I'd go ahead and spend more time figuring things out in simulation, aiming for something the team could actually demonstrate. I spent 3-4× more hours than expected on this project, largely because of that decision.

The second shift was more deliberate. After the hardware issues were resolved over Easter, Robin needed detection output for his state machine. I provided captured images and GPS coordinates via a defined interface. Building that interface was "quite expensive" — it required attitude correction for pitch and roll, altitude compensation, and dealing with significant single-estimate error. Robin and I discussed ideas as colleagues, working in close coordination during the integration period. Robin suggested the repulsive force concept for NFZ avoidance; I thought it was a great idea, got into it,

and developed it over several days of experimentation, testing edge cases like what happens when the dummy is placed very close to the no-fly zone boundary. This collaboration was the point where my role shifted from “solo CV developer” to “integration partner.” It happened late in the project, not at the start.

Drawing on Lencioni’s (2002) five dysfunctions framework, which we studied in our Professional Practice sessions with Graham, I now recognise that my role drift wasn’t purely conscientiousness — it was partly an expression of the first dysfunction, *absence of trust*. I was taking on work because I didn’t trust that gaps would be filled otherwise. I didn’t trust the process; I trusted my own output. That’s a reflex I brought from my background, where individual ownership is a core value — one person takes responsibility and delivers. I hadn’t examined whether that reflex was a strength or a failure mode in a five-person team. In this project, it was both. It kept the project alive through the hardware crisis, but it also meant I was a single point of failure: the number of core modules that only I could modify was too high.

[EBI] I should have recognised the role drift earlier and called a team meeting to renegotiate the split explicitly. The falsifiable signal I missed: if I’m taking on a task outside my original brief and can’t name the teammate it unblocks, I’m probably building for myself, not for the team. That test should have been running from week one.

1b. What I am most proud of

I want to be honest: what I’m most proud of is the simulation work and the CV detection-to-GPS estimation pipeline.

We had extremely limited real data. One flight video from Sid — that was it. That single video was all the real-world data available for the entire project. From that one video I had to evaluate models, test averaging techniques, deduce ground truth, retrain the detector, and calibrate the GPS estimation pipeline. Everything had to be squeezed from that one data source.

The GPS estimation problem was the core technical challenge. A single detection from a camera frame gives you a pixel coordinate. Converting that to a GPS position requires the drone’s altitude, its attitude (pitch and roll), the camera’s calibrated field of view, and a coordinate transform. Single estimates had significant error — several metres off — so I developed and tested multiple averaging techniques, each with different trade-offs. Inverse variance weighting, spatial clustering, Kalman filtering. I benchmarked all of them against ground truth I’d deduced from the video itself. This wasn’t textbook work; there was no established pipeline for “estimate a casualty’s GPS position from a moving drone camera with one flight video.” I had to figure out what would work, and the iteration cycle was: try an approach, run it against the video, measure the error, adjust, repeat.

The simulation framework was the other half. After the failed practice days — when the hardware provided to us wasn’t working and the whole team was disheartened — I decided that sitting around feeling frustrated wasn’t going to help. I’d go ahead and build a simulation detailed enough to anticipate real-world issues. I simulated GPS drift, camera shake, motion blur at different speeds, inference latency. I built a safety-critical layer with geofence enforcement and repulsive fields near the no-fly zone. I ran the simulation many times, observing closely, thinking through what could go wrong. Instead of holding problems in my head, I had a concrete environment to test against. The simulation could verify all twelve project requirements (R01–R12) end-to-end. I communicated this work to the team and they were happy with it.

[WWW] The fact that everything came together on the actual demo day without prior real-hardware testing of the full pipeline says something about the simulation’s quality. It was also somewhat lucky — I’d have much preferred to run the progressive test ladder step-by-step to build genuine confidence. But the simulation was detailed enough that when the real hardware finally worked, the software

running on it was already debugged.

The pride has a shadow, though. The simulation was very much my work. I spent 3–4× more hours than expected on this project, largely because of it. And that level of individual investment was also the behaviour that kept me from delegating more, from asking teammates to pair on tasks, from sharing ownership earlier. I was being productively solo at moments when the team might have benefited from a more shared response. What I’m proud of technically is also what I need to be honest about interpersonally.

1c. What I would focus on differently in future work

What this project forced me to confront is the distinction between *capability* and *enablement*. I came in believing the best engineer is the one who can fix the problem at midnight (reflection-in-action, Schön, 1983). I leave believing the best engineer is the one who makes the fix legible enough that the teammate who owns the module could have found it themselves.

The evidence: when Robin needed my CV output for his state machine, I provided captured images and GPS coordinates via a defined interface. Robin then integrated without asking me questions for two days. That was the first piece of work I did on this project that scaled without my presence — and it happened late, not early. The delay between my allocation and my first genuinely enabling contribution is the core lesson. I should have been building interfaces from week one, not accumulating private tools.

[APOLLO: what would you do differently? You’ve told me the problem (role drift, solo building, late interfaces). What’s the specific mechanism you’d use next time? A concrete rule, habit, or practice — not “communicate better.” For example: pair-programming budget? Weekly interface reviews? Named-beneficiary test before starting any new tool? Something with a falsifiable signal so you’d know if it’s actually working. Also: what’s your next step after this MSc — internship, job, further study? Need a real future context to anchor the mechanism.]

The sentence I’ve written in every group reflection since undergraduate is “I will communicate better.” It has never, once, changed my behaviour. Whatever mechanism I commit to next needs a falsifiable signal — something I can measure after four weeks and know whether it’s working or not. “Communicate better” isn’t falsifiable. A specific practice with a specific check is.

My team

2a. Team structure — how it formed and what changed

There were five of us: Apollo, Robin, Harish, Edward, and Zian. After the first team meeting we looked at all the tasks the project needed and split them up. Harish took hardware setup and communications between components. Zian took path planning. Robin took the flight state machine. Apollo and Edward shared computer vision — I delegated the CV model training to Edward while I focused on the detection-to-GPS pipeline, simulation, and integration.

[APOLLO: Was this structure discussed explicitly in the meeting, or did it just happen? Did someone suggest the split, or did people volunteer? Was there any disagreement about who took what — or did everyone just grab what felt natural?]

We deliberately did not name a single technical lead. The brief (R08) devolves autonomy-level choices to the team and we wanted distributed ownership. This felt sensible at the time.

Mapping this against Tuckman’s (1965) model, our Forming stage was brief — arguably too brief. Graham warns in Professional Practice that teams which spend “too little time forming, which means no building of trust in the team, no discussion of its aims or its goals, and just jumping straight into

the storming phase” produce the worst-case scenario where the team eventually needs to reform under pressure. That is approximately what happened to us. We split the work and started building. We did not spend time discussing how we would communicate, how we would handle disagreements, or what “done” meant for each module.

The structure then drifted — silently. The team had a while to work on everything before the flight days came. We did extensive preparatory work, but when it came to flight days we got quite disheartened and demotivated. The core issue was that we did not know what actually worked and what did not until we actually flew the drone. Many things could only be tested with real GPS signal, which was not available in the lab — anything requiring GPS coordinates could only be verified on flight days. We were working on hypotheses about what might work, but could not test those hypotheses without flying.

As practice flight days failed and hardware remained non-functional, the project’s centre of gravity migrated from the test field to my laptop — the only surface where forward progress was possible. I was running SITL simulation, writing test scripts, building the web ground station and dashboards, while Harish waited for working hardware and Robin waited for flight data to test his state machine against. Nobody declared this shift; it just accumulated. My role expanded from “CV specialist” to something much broader without anyone consciously choosing that.

In Lencioni’s (2002) terms, this silent drift is a textbook expression of Dysfunction 4, avoidance of accountability: without commitment to a clear plan of who owns what *this week*, there is nothing to measure against, and low standards prevail. We had committed to a structure in the first meeting, but we had not committed to a mechanism for detecting when that structure no longer matched reality.

[APOLLO: Was there ever a moment where someone said “wait, you’re doing too much” or “we need to re-split”? Or did the drift just go unchallenged? Did you ever raise it yourself?]

The structure changed most visibly after Easter, when the hardware finally started working. Robin needed detection output for his state machine, and we entered a period of close coordination. I provided captured images and GPS coordinates via a defined interface. Building this interface was expensive — it required attitude correction for pitch and roll, altitude compensation, and dealing with significant single-estimate error. Ideas were discussed between us as colleagues. This was the point where the team actually started functioning as an integrated unit rather than parallel streams, and it happened late — not by design, but because hardware failures had kept us apart.

One reading of this is that our team structure failed. Another is that it succeeded in the way that matters: when integration was finally possible, the interface between Robin’s state machine and my CV output worked. Robin consumed my output without needing me beside him. I conclude the structure worked not because it was well-designed at the start, but because we had the capacity to adapt when circumstances forced it. That capacity, I now believe, is more valuable than getting the initial design right.

[EBI] We should have revisited the structure formally at regular intervals, not only when external events forced it. A standing agenda item — “does our current structure still match the work?” — would have caught the drift before it became entrenched. The mechanism was simple; the discipline to use it was what we lacked.

2b. Most impactful elements of team working

The most impactful team-working element, measured by what it actually unblocked, was the close collaboration between Robin and me during the integration period after Easter.

Robin and I were the two proactive members of the team. After the hardware issues were sorted, Robin needed my CV output for his state machine. I provided captured images and GPS coordinates

via a defined interface. Robin suggested the repulsive force concept for no-fly zone avoidance — I thought it was a great idea, got into it, and developed it over several days of experimentation, testing edge cases like what happens when the dummy is placed very close to the NFZ boundary. Ideas were discussed between us as colleagues; neither of us could have produced the result alone. This kind of collaborative ideation — Robin proposes a concept, I implement and stress-test it, we iterate — was the most productive mode we found.

What made this collaboration work was a clear interface contract: Robin’s script handled flight decisions, my code handled perception and GPS estimation. We did not need to understand each other’s internals. The hours I spent writing interface documentation bought Robin significant unblocked integration time — he could wire my detection output into his mission script without asking clarifying questions. I had previously treated handoff docs as a cost paid for politeness. In fact they were the highest-leverage work I did on this project, measured by teammate-hours-unblocked per author-hour-invested.

[WWW] The progressive test ladder was the second most impactful element. I organised all flight tests into numbered steps (`tests/flight/0a_cube_commands.py` through `4_detect_and_center.py`), with a strict rule: never skip a step. Each step builds trust before adding risk. This was, in Graham’s (2026) risk management framework, a mitigation that reduced both likelihood and consequence. Due to limited flight time, the full test suite was never run on real hardware — but the ladder shaped how we approached testing. When we finally ran on the Pi during Easter, the delta between simulation and hardware turned out to be manageable: a few coordinate bugs and a corrupt calibration file. The system came up because the simulation and test infrastructure had caught everything else.

But I need to name the negative side too, because it was equally defining: **the hardware failures and their effect on team morale.**

The team had prepared thoroughly before the flight days came. I had an in-depth simulation, a progressive test ladder, multiple dashboards, test scripts for every subsystem. But when the practice flight days arrived, we were given faulty hardware. Things just did not work, and we were not sure what the issues were. The first two practice flight days were effectively lost. All the other teams had working hardware from the start; ours was not functional until Easter.

This was deeply demoralising. During those failed days, the whole team felt demotivated — including me. I had loads of things I wanted to run and test. But instead of just being frustrated, I decided to be proactive: I would go ahead and spend more time figuring things out in simulation, aiming for a thorough final product. The simulation was very useful and very detailed — I simulated camera blur, thought through everything that could go wrong, and ran the simulation many times observing closely. Instead of holding problems in my head, I had a concrete simulation to test against.

The frustration was compounded by organisational problems on flight days. The Wi-Fi signal was prioritised for teams that were actively flying, while the other teams were supposed to be tinkering on the ground. But without good Wi-Fi, we could not connect to our drone to test or configure anything. There were IP address and naming conflicts. The hardware that was not working was not our responsibility — it was provided to us.

[APOLLO: Any specific moment you remember from those failed flight days? What were you doing, who was there, what was the mood? Even a single sentence of concrete scene-setting would anchor this — e.g. “we sat in the field for two hours trying to get a connection while Team X flew loops above us.”]

[APOLLO: Were there any real disagreements within the team? The rubric rewards naming a conflict with a resolution mechanism. Did you and Robin disagree about anything — the state machine design, the interface format, the approach to NFZ avoidance? Even a mild one counts. If there genuinely were no disagreements, that is also a valid answer, but it limits the teamwork score.]

The turning point came during Easter, when the team sacrificed holiday time for two extra practice

days. On the first there were still some issues, but the hardware problems were finally resolved. Getting confirmation that the state machine worked on real hardware was the moment the team's energy shifted from frustration to focus. From that point, we had roughly one and a half functioning flight days before the performance demo.

In Tuckman's terms, those failed flight days forced us through an accelerated Storming phase — not a productive storming of ideas, but a collective crisis of confidence. The question was not “whose approach is better” but “does anything we built actually work.” That is a harder kind of storming because there is no resolution through debate; the only resolution is working hardware.

Everything came together on the actual demo day without prior real-hardware testing of the full pipeline. This worked because the simulation was thorough — but it was also somewhat lucky. The fact that it all came together pretty well says something about the quality of the preparation, but I would have much preferred the step-by-step confidence-building approach I had designed.

Looking at this through Lencioni's framework, the hardware crisis actually pushed us past some dysfunctions. The shared adversity — all five of us frustrated by the same external constraint — created a kind of forced trust. We stopped worrying about who owned what module and started worrying about whether anything worked at all. That is not the healthy path to Tuckman's Norming stage, but it was the path we had.

The complaint letter we sent to the course organiser about the hardware situation was, in a sense, the team's first genuinely collective action — all five names, one shared grievance, articulated in writing with measured factual tone. I led that communication because I had the clearest view of what the hardware failures had cost in engineering terms, and because I had the written evidence to back every claim. [APOLLO: Did Robin or anyone else review the letter before it went out? Did anyone push back on the tone? Did Steve respond? Any detail here strengthens the conflict-management evidence — this counts as external conflict management.]

[EBI] The flight day failures exposed a gap in our team's resilience planning. We had no contingency for “what if the hardware does not work on the day.” My simulation was a partial answer — it let me keep working — but it was not a team-level response. A team-level contingency would have been: “if hardware fails on practice day 1, here is a prioritised list of what each person does with their ground time.” We never wrote that list. Instead, most of the team waited for the hardware to work, and when it did not, they had nothing structured to do.

2c. What I would change about team structure

[APOLLO: What would you genuinely change? Below are three changes inferred from your context. But what does your gut say? Maybe it is something simpler — “I would pick teammates differently” or “I would push back on the hardware earlier” or “I would delegate less to Edward and more to Robin” or “I would insist on weekly check-ins from day one.” Your honest answer, even if blunt, will be stronger than anything I can infer.]

Three concrete structural changes, each tied to a specific failure I observed.

Change 1: Fortnightly structure review. I would institute a standing 15-minute check-in every two weeks with one question: “does our current task allocation still match the work that actually needs doing?” The silent drift — where I accumulated responsibilities nobody assigned me — would have been caught by any regular review of who was doing what. The falsifiable signal: zero surprises at integration time about who owns which module. Graham warns that teams which “don't bother to think about how they work, possibly because they hate talking about management, will have to spend even more time trying to manage the team.” That was us exactly.

Change 2: Front-loaded interface contracts. I would front-load interface contracts *before* parallel

work begins — a single early session on the data format between my CV output and Robin’s state machine would have saved integration time later. When Robin finally needed my output, building the interface was “quite expensive” — attitude correction, altitude compensation, averaging techniques for noisy single estimates. If we had agreed on the interface shape in the first weeks, I could have built to that contract from the start instead of retrofitting it.

Drawing on the down-selection tools from Graham’s (2025) innovation lecture, I would also propose using a structured decision framework for architecture choices, with explicit weighted criteria rather than informal discussion. Graham’s warning resonates: “if you skip the requirements analysis, you’ll most likely find that the solution selection takes much longer anyway as the team will find it harder to agree on what good is supposed to look like.”

Change 3: A hardware failure contingency owned by the whole team. This is the change most specific to our project. I would write, in the first week, a one-page document answering: “if the hardware does not work on flight day, what does each person do with their time?” My answer to that question was to build simulation — but that was my personal response, not a team plan. Edward and Zian, who would do what they were assigned but were not self-directed, had no equivalent fallback when the hardware failed. A team-level contingency would have given everyone structured work during the dead flight days, instead of leaving people waiting while I kept building.

[EBI] The deeper lesson, which connects to our Tuckman progression, is about how much time to invest in the team versus the technical work. Graham presents a model showing that good teams spend *more* time managing themselves early and *less* later, while poor teams do the opposite. We were the poor-team pattern: almost no time on team management early on, then managing the fallout of hardware failures and unclear ownership. If I could restart, I would advocate for one full session in the first week dedicated to nothing but team process: task allocation, interface contracts, communication cadence, a shared definition of “done” for each module, and a contingency plan for hardware failure. That session would have cost a few hours of technical work and saved days of frustration and rework.

Applying Lencioni’s antidotes directly: I would start with a personal histories exercise in the first meeting — sharing backgrounds, working styles, and what each person needs to do their best work. The point is not to become friends; it is to build enough trust that disagreements feel normal, not threatening. And I would institute what Lencioni calls “publication of goals and standards” — a shared document listing what each person has committed to deliver, by when, with what quality bar. The reason I did not do this is that it felt bureaucratic and paternalistic. The reason I should have is that, as Lencioni puts it, “peer pressure in the team, providing there’s a foundation of trust, is way more effective than a bureaucratic measurement system.” We had neither peer pressure nor a measurement system. We had me building things and hoping the team would notice.

The honest summary of 2c: I would change the team’s investment in its own processes, and the person I would need to convince first is myself, because my instinct is always to write code rather than write a decision log, and every lesson in this section is a lesson about that instinct being wrong.

My impact

3a. How my contributions drove the team forward

The honest thesis of this section: the simulation framework I built is the reason this project produced verifiable engineering output despite having faulty hardware for most of the project timeline. I want to defend that claim with mechanism, not assertion, because at Merit-level it reads as self-flattery and at Distinction-level it requires evidence.

The CV pipeline: detection to GPS estimation. My formal CV responsibility expanded into something far more complex than training a model. I built the full detection-to-GPS pipeline: take a

camera frame, run YOLOv8 inference, convert the pixel coordinates of a detection into a GPS position estimate for the dummy on the ground. That conversion is not trivial. The drone’s pitch and roll affect where the camera is pointing, so I had to correct for attitude. Altitude affects the scale of the image, so I had to compensate for that. And a single detection gives a GPS estimate with significant error — several metres off, sometimes more — so I developed and tested multiple averaging techniques: inverse-variance weighting, spatial clustering, Kalman filtering, each with different trade-offs in accuracy versus latency.

The constraint was data. We had one real flight video, provided by Sid. One video. I tested every averaging technique against that single recording. I deduced ground truth from the video itself — working backwards from known landmarks to establish where the dummy actually was — and benchmarked each technique against that ground truth. I also used the video to retrain the model: 16 labelled frames extracted at native resolution, combined with 300 synthetic images and 50 negatives. The retrained model ($\text{mAP}_{50} = 0.995$ on validation) was a drop-in replacement. I squeezed everything I could from extremely limited data.

The pipeline drove the team forward in a specific way: Robin’s state machine needed a GPS position for the detected target. Without a reliable estimate, his flight logic had nothing to navigate towards. The interface I provided — captured images plus estimated GPS coordinates, corrected for attitude and altitude — was what Robin consumed. Building that interface was “quite expensive,” as I described it at the time. But it meant Robin could develop his flight script against a defined contract without understanding the CV internals.

I also did thorough camera calibration in the lab — field of view and focal length measurements using a tape measure at known distances, validated against the DJI video using a click-to-calibrate tool I built. I created post-processing scripts for the DJI footage, calibration scripts for camera parameters, and video analysis tools — all designed to maximise learning from that single real flight recording.

The simulation framework — a Kolb cycle. This was the contribution that consumed the most hours and had the most ambiguous team impact. I want to walk through a full Kolb (1984) cycle, because I used it deliberately rather than retrospectively, and it is the piece of experiential learning that most changed my engineering practice during this project.

Concrete experience: Two practice flight days where the team had everything prepared — an in-depth simulation we wanted to test, a suite of test scripts we wanted to run — and the hardware did not work. We were given faulty hardware. Things just did not work, and we were not sure what the issues were. All the other teams had working hardware from the start. Our hardware was not functional until Easter. The first two practice flight days effectively did not count. The whole team felt demotivated — me included. I had loads of things I wanted to run and test.

Reflective observation: The team’s entire definition of “progress” had silently assumed hardware access we did not have. Robin could not test his state machine. Edward could not validate his interface. Many things could only be tested with real GPS signal, which was not available in the lab — anything requiring GPS coordinates could only be verified on flight days. We were working on hypotheses about what might work, but could not actually test those hypotheses without flying. The only person who was not fully blocked was me, because I had a laptop with SITL. That asymmetry was the data point that mattered.

Abstract conceptualisation: If the vision module exposed a single interface (`detect_in_image(frame)` returning `(found, x, y, conf)`) and auto-selected between the full model on laptop and the lightweight model on Pi, and if the config auto-detected its hardware, then the same state machine, geofence, and search pattern could run in both places. Anything that ran in simulation would, in principle, run on hardware. The key design decision was to make the vision module the ONLY file that touches CV. Everything else calls the same function. You can change the model, preprocessing, or backend without

touching any other file. That modularity was not aesthetic; it was risk mitigation. In Graham’s (2026) risk management framework, it reduced both the *likelihood* of a platform-incompatibility failure (by abstracting the backend) and the *consequence* (by isolating the blast radius to a single file).

Active experimentation: Instead of just being frustrated, I decided to be proactive: go ahead and spend more time figuring things out in simulation, aiming for a solid final product. The simulation grew to 2508 lines with simulated GPS drift, camera shake, CV throttle, and a TFLite backend path. I ran it thousands of times, observing closely. I simulated camera blur. I thought through everything that could go wrong. Instead of just holding problems in my head, I had a concrete simulation to run and test against. This helped me anticipate issues ahead of time rather than discovering them during the real mission. Robin suggested the repulsive force concept for no-fly zone avoidance — I thought it was a great idea, got into it, and developed it over several days of experimentation, testing edge cases like what happens when the dummy is placed very close to the no-fly zone boundary. The simulation could verify all R01–R12 requirements.

The Kolb cycle taught me something I had not previously articulated: simulation-first is not a personal preference. It is an engineering obligation when hardware access is unreliable. And the obligation has limits. The simulator could not predict real motion blur at 5 m/s with a real IMX296 sensor. It could not find the descent stall we eventually accepted as a SITL artefact. It could not tell us what GPS timing lag (100–200 ms latency, causing approximately 1 m error at 5 m/s) would do to a multi-pass target lock — that insight came from DJI video analysis, after the fact. The simulation’s value is bounded by the fidelity of its models, and I was honest about that only because I had been forced to be.

I spent 3–4× more hours than expected on this project, largely because of the simulation work. I communicated the simulation to the team and they were happy with it. I believe the simulation was almost ready to run in real mode, but to be confident about it we would have needed more flight days.

The progressive test ladder — risk assessment. I wrote progressive test scripts for empirical drone performance testing — blur effect at different speeds, detection range versus altitude, detection rate at various conditions. These formed a safety-critical test ladder designed for step-by-step confidence building before attempting a full mission:

1. **Bench commands** — test individual MAVLink commands on the bench, no propellers. Reduces the *likelihood* of a command-path failure reaching the field.
2. **Fly GPS waypoints, no CV** — the drone flies a simple pattern under script control with no perception involved. Isolates navigation risk from perception risk. The *consequence* of a CV bug is zero because CV is not running.
3. **Manual flight, passive detection** — pilot flies on RC, Pi runs camera and AI, logs detections, buzzer beeps, sends ZERO commands. Tests perception in real conditions with zero *consequence*: the pilot has full control, the system observes only.
4. **Autonomous search, logging only** — the full pipeline runs but detection triggers logging, not action. The *consequence* of a false positive is a log entry, not an unwanted descent.
5. **Full autonomous mission** — only attempted after four prior steps have passed.

In Graham’s (2026) risk assessment framework, each step reduces either the likelihood or the consequence of a failure mode, and no step introduces both navigation risk and perception risk simultaneously until step 5. The design is sound. The execution was not: due to limited flight time, we never ran the full ladder on real hardware. The residual risk of an unexecuted mitigation is the same as the residual risk of no mitigation at all.

Dashboards and monitoring tools. I built a passive watch dashboard for monitoring and detection during manual flight, and a diagnostics dashboard showing all drone subsystems at once with the ability to switch between different vision models live to compare performance. Both were built for the

Pi (headless, browser-based) so they could run on actual drone hardware without a screen attached. These tools were designed for the pilot and ground operator, not for me. [APOLLO: Did anyone other than you actually use these dashboards on flight day? Did the pilot use the passive watch? Any specific moment where a dashboard helped the team?]

Going with YOLO person detection. Edward did train a CV model as I had delegated. We did not end up using it. We went with a standard YOLO person detection model instead — the safe choice. The reasons: not enough flight days to collect realistic aerial training data, and we could not run proper experiments to validate a custom model. We did have trained models for detecting different items of interest, but in the end the YOLO person detection was safer given our limited real-world testing. In Graham’s risk framework, this reduced the *likelihood* of a detection failure on demo day at the cost of reduced specificity. The trade-off was correct given the constraints.

The flight day outcome. Everything came together on the actual flight day without prior real-hardware testing of the full pipeline. This worked because the simulation was thorough — but it was also somewhat lucky. The fact that it all came together on demo day without any real testing beforehand says something about the quality of the simulation work. Building on the success of the Easter days (about one and a half functioning flight days before the performance), the team managed to deliver a minimum viable product. It was not the standard we wanted — on the very first flight day we had already had a bunch of things ready to run. But we did the best we could given the hardware limitations, and what we delivered was solid given the constraints.

Evaluating my own effectiveness (M16.d). Against the criterion of “did my contributions unblock teammates or only advance my own work,” the judgement is mixed. The CV pipeline was genuinely team-enabling: Robin’s state machine consumed it directly. The simulation framework kept the project moving when hardware was unavailable. The handoff documentation worked: Robin integrated without clarifying questions. But the simulator itself (2508 lines) was edited by zero teammates across the entire project. The unit tests I wrote (127 functions across 6 files) were run by zero teammates. The cause is not that these artefacts were bad — it is that I built them as solo infrastructure rather than collaborative tools. I was effective at producing artefacts but ineffective at producing shared ownership, and those are different kinds of effectiveness.

Evaluating my team’s effectiveness (M16.d-team). We did the best we could with what we had. We could verify all 12 requirements in simulation and demonstrated a minimum viable product on real hardware. The gap was caused by hardware unavailability (external, not our fault), coordination gaps (internal, our responsibility), and insufficient investment in team processes (internal, largely my responsibility as the person who set the technical pace). Our Tuckman progression — brief Forming, extended frustration, late integration, compressed performance — reflects a team that invested too little time in managing itself, exactly the pattern Graham warns about. We treated process as overhead rather than infrastructure.

Communication methods evaluated (M17.d). I used several communication methods across the project:

1. **Technical handoff documentation** (Robin) — pipeline docs for the CV-to-state-machine interface, with exact CLI commands, JSON schema, parsing examples
2. **Browser-based ground station** (the pilot and team) — headless web dashboard accessible from any device at the field
3. **Complaint letter** (Steve, the course organiser — a non-engineering audience) — formal, factual, deliberately restrained, signed by all five members
4. **Field reference card** — copy-paste-ready terminal commands for flight day with no internet access
5. **Progressive test scripts** (team) — numbered scripts that enforce a step-by-step confidence-

building process

Against the criterion of whether the recipient could act without coming back with clarifying questions: the handoff documentation worked — Robin integrated without questions. [APOLLO: Did anyone use the browser dashboard on flight day? What was Steve’s response to the complaint letter? Did anyone use the field reference card?]

The lesson: communication effectiveness is not about clarity — it is about *cost asymmetry*. The hours I spent writing handoff docs bought Robin many hours of unblocked integration time. The right strategy is to document for future-proofing but pair-program for urgent unblocking — and I should have been explicit about which mode I was in, rather than defaulting to documentation because it is my comfort zone.

[APOLLO: For M17.c (non-technical audience), the complaint letter to Steve is the strongest candidate. Can you confirm: (1) Steve is non-technical / not an engineer? (2) What adaptations did you make to the language? (3) Did Robin or anyone else review it before sending? (4) What was the response?]

3b. What I would add or re-focus on in future projects

The specific gap I want to name: I mistook technical leadership for team leadership. I now see they are orthogonal — you can build great tools that do not enable anyone — and in this project the gap showed up as a simple fact: the core simulation and mission files were edited by zero teammates. That is not an ownership success; it is an ownership failure. I owned too much, shared too little, and called it diligence.

Graham’s (2026) risk management lecture introduces three pillars of obligation: moral, economic, and legal. Applying the economic lens: the team’s “currency” is shared capability, and a massive codebase that only one person can edit is an illiquid asset. Code that no one else can maintain is not a team asset; it is a personal portfolio piece that happens to live in a shared repository.

Three mechanisms for future refocusing:

1. Build for named users from the start. Before starting any new tool, I will name at least one teammate who will use it and document their name in the commit message. The test I should have been running: if I had built the simulator explicitly for Robin to test his flight logic without a drone, would I have designed it differently? Yes — I would have made the keyboard controls match his flight-script commands, not mine. The tool would have been less elegant and more useful. A tool built for “the team” is often a tool built for nobody.

2. Execute the test ladder, not just design it. I am proud of the progressive test ladder as a design. I am not proud that we never ran it. Next time, I would build the test ladder AND build the project schedule around it — not as an optional extra, but as the critical path. In Graham’s risk framework, an unexecuted mitigation carries the same residual risk as no mitigation at all. The would-have-liked-to is not a plan.

3. Earlier integration, not just late collaboration. Robin and I worked well together once integration was needed, and it went well. But we started integrating late — after Easter, under time pressure. If we had integrated earlier, even with stub data, we would have found interface mismatches sooner and built shared understanding of the codebase earlier. Integration is not a phase; it is a practice.

[APOLLO: what would you add or refocus? specific behaviour/process? For example: would you change how you delegated CV training to Edward? Would you push harder for more flight days earlier? Would you set up regular check-ins or stand-ups? Anything about your own working style you would deliberately change — not just “share more” but a concrete mechanism?]

Applying Graham’s risk framework to these mechanisms: Mechanism 1 reduces the *likelihood* of building unused tools. Mechanism 2 reduces the *consequence* of hardware surprises (the test ladder becomes the critical path, not a supplement). Mechanism 3 reduces both — early integration transfers knowledge (likelihood) and creates backup capability (consequence). The residual risk: even with these mechanisms, I may still default to solo work under time pressure. That residual risk is not zero, and I acknowledge it rather than pretending the mechanisms are sufficient.

Applying Graham’s ALARP principle: is my current approach to team contribution “as low as reasonably practicable” in terms of team risk? No. If I had been ill during the last two weeks of the project, the team would have lost access to the CV pipeline, the simulation framework, the ground station, the test ladder, and all monitoring tools. That bus factor of one is intolerable. The mitigations I have proposed reduce the risk to tolerable. But they require me to accept slower initial progress in exchange for faster recovery from disruption, and that trade-off is the one I find hardest to make.

My support

4a. What others did that helped me work more effectively

I want to name specific people and specific mechanisms, because a generic list of thanks says nothing about what I learned.

Robin’s collaboration and ideas. Robin was the teammate who most changed how I worked. After the hardware was sorted, Robin needed detection output for his state machine, and we entered a period of close coordination. But Robin didn’t just consume my output — he contributed ideas that shaped the system. He suggested the repulsive force concept for NFZ avoidance, which I thought was a great idea and developed over several days of experimentation. Ideas were discussed between us as colleagues, not handed down. That dynamic — two people iterating on a shared problem, each bringing different expertise — was what the whole team should have had from the start.

[APOLLO: Can you give a specific example of Robin pushing back on something you proposed, where you ended up changing your approach? The planner mentions a “wk17 FSM pushback” but it’s unverified. Any real instance of disagreement-then-resolution would be very valuable. Even something small like “Robin said my interface was too complicated and I simplified it.”]

One reading of Robin’s input is that it was just one suggestion among many. Another reading is that it represents the kind of collaboration that produces better results than either person alone. I conclude the second, and the evidence is that the repulsive force system — Robin’s concept, my implementation — was one of the most robust safety features in the final system. The NFZ avoidance worked because it was designed by two people with different perspectives on the problem.

Harish’s hardware work. Harish was responsible for hardware setup and communications between components. [APOLLO: Can you give a specific example of something Harish did that helped you? Did he set up the Pi-to-Cube connection that you then built on? Did he debug a wiring issue? Did he help during the Easter sessions when hardware finally started working? Any concrete moment where his work unblocked yours?]

Edward’s model training and perspective. Edward did train a CV model as I’d delegated. We didn’t end up using it — we went with standard YOLO person detection, playing it safe, because we didn’t have enough flight days to collect realistic aerial training data. But Edward’s work wasn’t without value: having someone else train a model forced me to define what “good enough” meant for detection performance, which fed back into my own pipeline design. [APOLLO: Did Edward ever say something specific — even a casual remark — that changed how you thought about a problem? The old draft had a fabricated “pyserial breakthrough” story. Is there a real moment? Even something like “Edward pointed out that testing on laptop doesn’t mean it works on Pi” would be enough if true.]

The team’s collective effort during Easter. After two failed practice days, the team came together during Easter holiday and got the hardware working. On the first Easter day there were still issues, but they managed to fix the hardware problems and get something going — finally getting confirmation that the state machine worked. After that confirmation, we could start building on top of it. The team sacrificed their holiday to make this happen. That collective effort, even when everyone was demotivated, is what made the demo day possible.

[APOLLO: Who specifically was at the Easter sessions? What did each person do? Who fixed the hardware issue? “The team” is vague — naming who did what would strengthen this significantly. Was Zian there? Was Harish the one who sorted the hardware?]

Sid’s flight video. Sid, the technical assistant, provided the single DJI flight video with GPS telemetry that anchored my entire CV pipeline. That one video was all the real flight data available for the project. I used it for model evaluation, GPS estimation benchmarking, FOV calibration, and retraining. One video, multiple downstream outputs. That contribution doesn’t show up in commit logs but it defined what was technically possible.

4b. What I did that helped others work more effectively

I will put specific episodes on record, because “I gave constructive feedback when needed” is the single most common Merit-ceiling sentence in UK engineering reflection, and I’m not going to write it.

Episode 1: Building Robin’s integration interface.

After the hardware issues were sorted, Robin needed detection output for his state machine. I provided captured images and GPS coordinates via a defined interface. This interface was not trivial — it required attitude correction for pitch and roll, altitude compensation, and dealing with significant single-estimate error. I built it specifically so Robin’s state machine could consume vision data without touching my CV code.

The result: Robin integrated without asking me a single clarifying question for two days. I had communicated the simulation work to the team and they were happy with it. The interface worked because I’d anticipated what Robin would need rather than just exposing whatever internal data structure I happened to use.

The reflection: building that interface was the highest-leverage work I did on the project, measured by teammate-hours-unblocked per author-hour-invested. I had previously treated integration work as a cost paid for politeness. In fact it was more impactful than most of the solo features I built. That reframing — from “overhead” to “highest leverage” — is one of the core belief changes I take from this project.

Episode 2: The complaint letter on behalf of the team.

The team had been provided with faulty hardware. All other teams had working hardware from the start. Our hardware wasn’t functional until Easter. The first two practice days effectively didn’t count, and Easter shouldn’t really count as a work day since we sacrificed holiday time. Rather than letting frustration dissipate as corridor complaints, I channelled it into a formal written artefact — a complaint letter to the course organiser documenting what had happened and its impact on the team’s ability to meet learning outcomes.

I led this communication because I had the clearest view of what the hardware failures had cost in engineering terms. The letter’s tone was deliberately measured and factual — documenting what happened, not assigning personal blame.

[APOLLO: Critical questions for this episode: (1) Did you share the draft with the team before sending? (2) Did anyone edit it or push back on the tone? (3) Was it signed by everyone or just you? (4) What

was Steve’s response? (5) Did it change anything (e.g., assessment criteria adjusted)? Without these answers, this episode is thin. With them, it becomes strong evidence for both conflict management and non-technical audience communication (M17.c).]

The mediating action was *externalising an internal frustration*: I turned a team tension into a written artefact that could be discussed and edited collectively, rather than letting it fester as unspoken resentment. I’ve noticed that I can only handle confrontation when it’s mediated by writing, and this is a pattern across my life, not just this project. Whether that’s a cultural habit or a personal one, I can’t fully separate. But I now recognise it as a tool with limitations: it works when there is time for iteration, and it fails when the issue needs a quick conversation.

Episode 3: Tools and infrastructure for the team.

The quieter half of my support work was building infrastructure the team could use without my presence:

- Progressive test scripts — numbered and ordered so nobody could skip a step, designed for step-by-step confidence building before attempting anything risky
- Browser-based dashboards and monitoring tools — headless-capable, so anyone with a browser could see what the drone was doing without needing a screen attached to the drone
- A field reference card for no-internet flight day use — written after the first cancelled day when I realised we’d be standing in a field with no internet
- Camera calibration data — FOV and focal length measurements done in the lab so the GPS estimation pipeline would have real numbers
- Video analysis tools — built to maximise learning from the single DJI flight video
- The simulation framework itself — which served as the team’s backup when hardware was unavailable, and which could verify all R01–R12 requirements

These artefacts were not glamorous. They didn’t train models or fly drones. But they were the substrate that let the project produce results when hardware didn’t cooperate.

[APOLLO: CRITICAL GAP — For the Insight 70+ gate, you need a specific feedback-to-others episode where you gave a named teammate specific feedback about their work and their behaviour visibly changed as a result. The episodes above are about building things for others, which is good but not the same as giving feedback. Can you recall:]

- [APOLLO: A moment where you told Robin something specific about his code/approach and he changed it?]
- [APOLLO: A time you pointed out a bug or design issue to Edward and he started checking for that pattern?]
- [APOLLO: A time you gave Harish or Zian feedback on their deliverable and they improved it?]
- [APOLLO: Even something like “I showed Edward how I structured test scripts and he started organising his own work similarly” — any peer development moment?]

[APOLLO: Without a real feedback episode, the Insight criterion is capped at 68. This is the single biggest gap in the goldmine right now.]

What I leave believing

I came to this project as someone whose reflex under failure was “one person delivers.” I leave believing its near-inversion: the engineer who delivers alone is one step away from being the single point of failure in whatever they build.

The close collaboration with Robin during integration, and the fact that the system worked on flight day, told me that shared interfaces are more valuable than solo features. The simulation kept the project alive when hardware failed — and I’m proud of that. But the simulation was also something

only I could maintain, and that fragility is the lesson I carry forward.

The team did a great job given the circumstances. We did the best we could with the hardware we were provided. Our limitations were not of our own making — we were given non-functional hardware and facilities that constrained what we could test. Despite all of this, we produced a working system that demonstrated on demo day. Building on the success of that one and a half functioning days before the performance, the team came up with something decent for the demo — a minimum viable product. Not the standard we wanted; on the very first flight day we'd already had much more ambitious things ready to run. But we did the best we could, and what we delivered was solid given the constraints.

Applying Tuckman's model to the full arc: we Formed briefly at the first meeting, struggled through a long period of frustration as hardware failed, reached a fragile Norming during Easter when things finally started working, and compressed our Performing into the demo day. We are now in Tuckman and Jensen's (1977) Adjourning stage. I feel the tension they describe — not because the project was a resounding success, but because I can see the team we could have been if we'd invested in trust-building from the start instead of trust-avoidance through code.

What remains unresolved: I don't yet know how to build trust quickly in a team of strangers. The mechanisms I've proposed — pair-programming budget, named-beneficiary test, documentation paired with conversation — are process scaffolds, not relationship tools. The relationship part, the part Lencioni says is the foundation of everything, I still find uncomfortable, and I suspect I will continue to find it uncomfortable. But I know, now, that the discomfort is data, not a deficiency, and that ignoring it is more expensive than sitting with it. That is the lesson I will carry.

[APOLLO: The closing needs your actual next step — internship, job, further study — and a falsifiable signal tied to it. Something like: “In my first six weeks at [company/lab], I will track whether at least one teammate modifies a module I own without asking permission. If yes, I've started enabling. If no, I'm still the person who writes files only he can read.” But it must reference something real.]

References

- Bennett, M.J. (1986). A Developmental Approach to Training for Intercultural Sensitivity. *International Journal of Intercultural Relations*, 10(2), 179–196.
- Graham, M. (2025). The Five Dysfunctions of a Team [Lecture video]. AENGM0074 Professional Practice, University of Bristol.
- Graham, M. (2025). Innovation Methods: Down-Selection Tools [Lecture]. AENGM0074 Professional Practice, University of Bristol.
- Graham, M. (2026). Building Cultural Competence [Lecture video]. AENGM0074 Professional Practice, University of Bristol.
- Graham, M. (2026). Assessing and Managing Risk [Lecture video]. AENGM0074 Professional Practice, University of Bristol.
- Hatton, N. and Smith, D. (1995). Reflection in teacher education: towards definition and implementation. *Teaching and Teacher Education*, 11(1), 33–49.
- Kolb, D.A. (1984). *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice-Hall.
- Lencioni, P.M. (2002). *The Five Dysfunctions of a Team: A Leadership Fable*. San Francisco: Jossey-Bass.

- Lencioni, P.M. (2005). *Overcoming the Five Dysfunctions of a Team: A Field Guide*. San Francisco: Jossey-Bass.
- Schön, D.A. (1983). *The Reflective Practitioner: How Professionals Think in Action*. New York: Basic Books.
- Shore, L.M., Randel, A.E., Chung, B.G., Dean, M.A., Ehrhart, K.H. and Singh, G. (2011). Inclusion and Diversity in Work Groups. *Journal of Management*, 37(4), 1262–1289.
- Tuckman, B.W. (1965). Developmental Sequence in Small Groups. *Psychological Bulletin*, 63(6), 384–399.
- Tuckman, B.W. and Jensen, M.A.C. (1977). Stages of Small-Group Development Revisited. *Group & Organization Studies*, 2(4), 419–427.